VL

STUDENT LAB WORKBOOK

MLOps & Model Deployment

Six live labs and a capstone — from a model file to a live, monitored deployment.

MLOps & Model Deployment · 1-credit · 15 hours
Vel Tech · UG (CSE / AI & DS / IT)

Instructor **Dr. Venkateshwaran Loganathan**
July 2026

CONTENTS

Lab Manual

GETTING STARTED

How to use this manual & setup

This workbook holds every hands-on lab for **MLOps & Model Deployment**. You build ONE project across three days and take it all the way to a live, public URL. Work in order — each lab extends the last.

Before you start

- A laptop with **Python 3** and **Docker** installed.
- Free accounts: **GitHub** (public repo + Actions) and **Hugging Face** (free Docker Space for deployment). Render is an alternative host.
- Clone the sample project, create a virtual environment, and install requirements:

```
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python make_data.py && python train.py            # data + a first tracked model
```

The through-line: train → track (MLflow) → serialize → serve (FastAPI) → containerize (Docker) → CI/CD (GitHub Actions) → deploy (free host) → monitor (drift). Every lab is one hop on this line.

LAB 1 · DAY 1

Lab 1 — Experiment Tracking with MLflow

Maps to: Session 3 (Day 1 — Foundations & Experiment Management) **Estimated time:** 2.5 hours **Format:** live-coding + pair programming + class leaderboard

Objective

Stop tracking experiments in your head (or in a messy notebook). By the end you will:

- Log **parameters, metrics, and artifacts** for every training run to MLflow.
- **Compare** many runs visually in the MLflow UI.
- **Register** your best model in the MLflow **Model Registry** and give it a **version**.
- Understand why this is the backbone of reproducible ML.

What you'll build

Multiple tracked runs of the loan-default model, a side-by-side comparison in the MLflow UI, and a registered model `loan-default-model` with at least version 1 promoted to a stage.

Setup

```
cd sample-project
source .venv/bin/activate          # Windows: .venv\Scripts\activate
python make_data.py                # if you haven't already
```

MLflow is already configured in `train.py` to use a **local SQLite backend**:

```
MLFLOW_TRACKING_URI = "sqlite:///mlflow.db"   # a file in your project folder
EXPERIMENT_NAME = "loan-default"
```

No server, no cloud — just a local file.

Step by step

Step 1 — Run your first tracked experiment (live-code together)

```
python train.py
```

You'll see a **Run ID** and metrics printed. Behind the scenes MLflow wrote to `mlflow.db` and a `mlruns/` folder. Open the code and find the three logging calls — read them out loud with your pair:

```
mlflow.log_params({...})     # the knobs you turned
mlflow.log_metrics({...})    # how well it did
mlflow.sklearn.log_model(...)# the actual trained model (an artifact)
```

> **Concept:** a *run* = one execution with its params, metrics, and artifacts. An *experiment* = a named group of runs. Tracking makes ML **reproducible** — you can always answer "what produced this number?"

Step 2 — Launch the MLflow UI

In a **second terminal** (keep it open all lab):

```
cd sample-project
source .venv/bin/activate
mlflow ui --backend-store-uri sqlite:///mlflow.db
```

Open **http://127.0.0.1:5000**. Click the `loan-default` experiment. You should see one run. Click it and explore Parameters, Metrics, and Artifacts (your model is under `model/`).

Step 3 — Sweep hyperparameters (produce runs to compare)

`train.py` accepts command-line arguments. Run at least **four** combinations:

```
python train.py --n_estimators 50  --max_depth 3
python train.py --n_estimators 200 --max_depth 5
python train.py --n_estimators 300 --max_depth 10
python train.py --n_estimators 500 --max_depth 20
```

Refresh the UI. Select all runs (tick the checkboxes) → click **Compare**. Sort by `roc_auc`.

Talk with your pair: which params gave the best `roc_auc`? Did more trees always help?

Step 4 — Log an extra artifact (hands-on edit)

Let's log a confusion-matrix image so it travels with the run. Add this near the end of `main()` in `train.py`, right after `mlflow.log_metrics(metrics)`:

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

fig, ax = plt.subplots()
ConfusionMatrixDisplay.from_predictions(y_test, preds, ax=ax)
fig.savefig("confusion_matrix.png")
mlflow.log_artifact("confusion_matrix.png")
plt.close(fig)
```

> `matplotlib` isn't in requirements. Install it just for this step: `pip install matplotlib`. (In real projects you'd pin it in requirements.txt.)

Re-run `python train.py` and view the image under the run's **Artifacts** tab.

Step 5 — Register your best model (Model Registry)

Two ways — do it via the UI (fastest):

1. In the UI, open your **best** run (highest `roc_auc`).
2. Open the **Artifacts** panel, click the `model` folder, then **Register Model**.
3. Name it `loan-default-model`. This creates **Version 1**.
4. Go to the **Models** tab (top nav) → open `loan-default-model` → set the stage of Version 1 to **Staging** (dropdown next to the version).

Now register a *second* run's model under the **same name** → it becomes **Version 2**. You now have a versioned history you can promote/rollback.

> **Concept:** the Model Registry separates "an experiment produced a model" from "*this* model is the one we serve." Stages (None → Staging → Production → Archived) are how teams promote and roll back.

Step 6 — Load a model back from the registry (prove it works)

```
# scratch.py — run with: python scratch.py
import mlflow
mlflow.set_tracking_uri("sqlite:///mlflow.db")
model = mlflow.sklearn.load_model("models:/loan-default-model/Staging")
print("Loaded from registry:", type(model))
```

```
python scratch.py
```

Checkpoint (raise your hand)

Show the instructor: 1. The **Compare** view with **4+ runs** and metrics. 2. The **Models** tab showing `loan-default-model` with **at least 2 versions**, one in **Staging**. 3. A run whose **Artifacts** include the confusion-matrix image.

"Break it" challenge

Deliberately break reproducibility, then fix it:

1. In `train.py`, change `random_state=42` to `random_state=None` in the train/test split **and** the classifier. Run twice with identical params.
2. In the UI, compare the two runs — the metrics differ even though params look identical. **Why is this dangerous?** (You can't reproduce a result.)
3. Fix it by restoring the seeds. Bonus: add `mlflow.log_param("random_state", 42)` so the seed is *tracked* — an untracked seed is a hidden parameter.

Interactive element — Class Leaderboard

- Each pair posts their **best** `roc_auc` (and the params that produced it) on the shared class board (whiteboard or a shared doc).
- Rules: same dataset (`make_data.py`, seed fixed), you may only tune params exposed in `train.py` (add `min_samples_leaf` if you want more knobs).
- Prize categories: **Highest ROC-AUC**, **Best AUC with ≤ 50 trees** (efficiency), and **Most runs logged** (thoroughness). Winners screen-share their MLflow Compare view.

Common pitfalls

- `Run python make_data.py first` — you skipped data generation.
- **UI shows no experiments** — you launched `mlflow ui` *without* `--backend-store-uri sqlite:///mlflow.db`, so it's reading the wrong store. Stop it and relaunch with the flag, from the project folder.
- **Port 5000 in use** (common on macOS AirPlay) — run `mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5001` and open `:5001`.
- **Two terminals** — one runs `train.py`, the other keeps the UI alive. Don't Ctrl-C the UI.
- **Registered the wrong run** — check the `roc_auc` before clicking Register.

LAB 2 · DAY 2

Lab 2 — Serving with FastAPI

Maps to: Session 5 (Day 2 — Packaging & Serving) **Estimated time:** 2 hours **Format:** live-coding + pair programming + Break-it validation game

Objective

Turn a saved model file into a **real web API** that anything can call over HTTP. By the end you will:

- Serve `model.joblib` behind **FastAPI + uvicorn**.
- Define request/response contracts with **Pydantic** and get automatic validation.
- Test the API three ways: **Swagger UI**, **curl**, and a **Python client**.
- Understand `422 Unprocessable Entity` and why validation protects your model.

What you'll build

A running service with: - `GET /health` — liveness check. - `POST /predict` — validated single prediction. - `POST /predict_batch` — you'll add this (many predictions in one call).

Setup

```
cd sample-project
source .venv/bin/activate
python make_data.py && python train.py    # ensure model.joblib exists
```

Step by step

Step 1 — Run the skeleton and meet Swagger (live-code together)

```
uvicorn app.main:app --reload
```

Open **http://127.0.0.1:8000/docs**. This is **Swagger UI** — auto-generated from your Pydantic models. FastAPI gives you interactive docs for free.

- Expand **GET /health** → **Try it out** → **Execute**. You should get `{"status":"ok", ...}`.
- Expand **POST /predict** → **Try it out**. The example payload is pre-filled → **Execute**. You should get a probability, a `prediction` (0/1), and a `label`.

> `--reload` restarts the server whenever you save a file. Keep this terminal open all lab.

Step 2 — Read the contract (understand before you change)

Open `app/main.py` with your pair and find the two schemas:

```
class LoanApplication(BaseModel):
    age: int = Field(..., ge=18, le=100)
    income: float = Field(..., gt=0)
    credit_score: int = Field(..., ge=300, le=850)
    ...

class PredictionResponse(BaseModel):
    default_probability: float
    prediction: int
    label: str
```

Every `Field(...)` with `ge` / `le` / `gt` is a **guardrail**. If a caller violates it, FastAPI rejects the request with **422** *before your model ever runs*. This is how you keep garbage out.

Also note `row_to_frame()` from `src/features.py` — it builds the feature row in the **exact same column order** used in training. This single shared function is your defence against **train/serve skew**.

Step 3 — Call it from the terminal with curl

Open a **second terminal**:

```
# health
curl http://127.0.0.1:8000/health

# a prediction
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"age":45,"income":40000,"loan_amount":30000,"credit_score":580,"employment_years":
2,"num_prior_defaults":3}'
```

That second applicant is high-risk (big loan vs income, low score, prior defaults) — notice the higher `default_probability`. Try a low-risk applicant and watch the probability drop.

Step 4 — Call it from Python (a real "client")

```
# client.py
import requests

payload = {
    "age": 45, "income": 40000, "loan_amount": 30000,
    "credit_score": 580, "employment_years": 2, "num_prior_defaults": 3,
}
r = requests.post("http://127.0.0.1:8000/predict", json=payload)
print(r.status_code, r.json())
```

```
python client.py
```

Step 5 — Add a batch endpoint (your turn — complete the TODO)

Find the `# TODO (LAB 2)` marker at the bottom of `app/main.py`. Add this above it:

```
from typing import List

class BatchRequest(BaseModel):
    applications: List[LoanApplication]

class BatchResponse(BaseModel):
    predictions: List[PredictionResponse]


@app.post("/predict_batch", response_model=BatchResponse)
def predict_batch(batch: BatchRequest):
    model = get_model()
    results = []
    for application in batch.applications:
        features = row_to_frame(application.model_dump())
        proba = float(model.predict_proba(features)[0, 1])
        prediction = int(proba >= 0.5)
        results.append(
            PredictionResponse(
                default_probability=round(proba, 4),
                prediction=prediction,
                label="default" if prediction else "repay",
            )
        )
    return BatchResponse(predictions=results)
```

Save (server auto-reloads). Refresh `/docs` — **POST /predict_batch** appears. Try it with two applications in the list.

Step 6 — Run the tests

```
pytest -q
```

The starter tests already cover `/health`, a valid prediction, and two validation-failure cases.

Checkpoint (raise your hand)

Show the instructor: 1. `/docs` open with a **successful** `/predict` response. 2. A **curl** call returning JSON from your terminal. 3. Your new **`/predict_batch`** returning a list of predictions. 4. `pytest -q` green.

"Break it" challenge — make the API say 422

Feed it bad input on purpose and observe how validation protects you:

1. **Missing field** — remove `credit_score` from the payload in `/docs` and Execute. → **422**, and the response tells you exactly which field is missing.
2. **Wrong type** — send `"credit_score": "seven hundred"`. → **422** (not an int).
3. **Out of range** — send `"age": 5` (below `ge=18`) or `"credit_score": 9999` (above `le=850`). → **422**.
4. **Extra nonsense field** — add `"favourite_colour": "blue"`. What happens? (By default it's ignored.) Discuss with your pair: should the API reject unknown fields? Try adding `model_config = {"extra": "forbid"}` to `LoanApplication` and re-test.

For each, note the HTTP status and read the error body. **This is the whole point of Pydantic** — invalid data never reaches your model.

Interactive element — "Break the API" relay □

Pairs take turns at the projector. Each pair must craft **one payload** that triggers a **422** for a *different reason* than any pair before them (missing field, wrong type, below-min, above-max, null value, etc.). If you repeat a reason already used, you're out. Last pair standing wins. The instructor keeps a running list of "reasons discovered" on the board.

Common pitfalls

- `uvicorn: command not found` — venv not activated, or run `python -m uvicorn app.main:app --reload`.
- `Model file 'model.joblib' not found` **(503)** — run `python train.py` first.
- `ModuleNotFoundError: app` — run uvicorn from the **project root** (the folder that contains the `app/` directory), not from inside `app/`.
- **Address already in use** — a previous uvicorn is still running; Ctrl-C it or use `--port 8001`.
- **Edited the file but nothing changed** — you're not running with `--reload`, or you edited a different copy. Watch the terminal for the reload message.
- **Sending form data instead of JSON** — always include `-H "Content-Type: application/json"` with curl.

LAB 3 · DAY 2

Lab 3 — Containerization with Docker

Maps to: Session 6 (Day 2 — Packaging & Serving) **Estimated time:** 2 hours **Format:** live-coding + pair programming + "it works on my machine" challenge

Objective

Package the API so it runs **identically anywhere** — your laptop, a classmate's, or a free cloud host. By the end you will:

- Read and complete a **Dockerfile**.
- **Build** an image and **run** a container.
- **Map ports** from container to host and hit the containerized API.
- Manage images/containers (`ps`, `logs`, `stop`, `rm`, `images`).

What you'll build

A Docker image `loan-default` that, when run, serves the FastAPI app on a port you map.

Setup

- **Start Docker Desktop** and wait until it says "Docker is running".
- Verify: `docker --version` and `docker run hello-world`.
- Ensure `model.joblib` exists (the image copies it in):

```
cd sample-project
source .venv/bin/activate
python make_data.py && python train.py
docker --version
```

Step by step

Step 1 — Read the Dockerfile line by line (live-code together)

Open `Dockerfile`. With your pair, explain each instruction out loud:

```
FROM python:3.11-slim              # tiny base image with Python
WORKDIR /app                       # working dir inside the container
COPY requirements.txt .            # copy deps FIRST (layer caching)
RUN pip install --no-cache-dir -r requirements.txt
COPY src ./src                     # then the code + model
COPY app ./app
COPY model.joblib ./model.joblib
ENV PORT=8080
EXPOSE 8080                        # documents the port the app listens on
CMD uvicorn app.main:app --host 0.0.0.0 --port ${PORT}
```

> **Why copy `requirements.txt` before the code?** Docker caches each layer. If only your code changes, Docker reuses the (slow) pip-install layer instead of reinstalling everything. Order matters for build speed.

> **Why `--host 0.0.0.0`?** Inside a container, `127.0.0.1` means "only the container itself." You must bind to `0.0.0.0` so traffic from outside the container can reach the app.

Step 2 — Build the image

```
docker build -t loan-default .
```

- `-t loan-default` tags (names) the image. The `.` is the build context (current folder).
- Watch the steps run. The first build is slow (downloads base + installs deps); later builds are fast thanks to caching.

List your images:

```
docker images | grep loan-default
```

Step 3 — Run the container with a port mapping

```
docker run -p 8080:8080 loan-default
```

`-p HOST:CONTAINER` maps **host port 8080 → container port 8080**. Open another terminal:

```
curl http://127.0.0.1:8080/health
curl -X POST http://127.0.0.1:8080/predict \
  -H "Content-Type: application/json" \
  -d '{"age":35,"income":60000,"loan_amount":15000,"credit_score":700,"employment_years":
8,"num_prior_defaults":0}'
```

Also open **http://127.0.0.1:8080/docs** — the same Swagger UI, now served from inside the container.

Step 4 — Prove the mapping (change the host port)

Stop the container (Ctrl-C), then run it on a **different host port**:

```
docker run -p 9000:8080 loan-default
```

Now the API is at **http://127.0.0.1:9000/health** even though the app *inside* still listens on 8080. Discuss with your pair: the left number is yours to choose, the right must match what the app listens on.

Step 5 — Run detached and manage the container

```
# run in the background, name it
docker run -d --name loan-api -p 8080:8080 loan-default

docker ps                 # see it running
docker logs loan-api      # view the app logs
docker stop loan-api      # stop it
docker rm loan-api        # remove the container
docker images             # list images
```

Step 6 — Simulate the cloud port (preview of Lab 5)

Hugging Face Spaces serves on port **7860** and passes it via `$PORT`. Because our `CMD` reads `${PORT}`, the *same image* adapts:

```
docker run -e PORT=7860 -p 7860:7860 loan-default
curl http://127.0.0.1:7860/health
```

You just proved your image is host-agnostic — exactly what Lab 5 needs.

Checkpoint (raise your hand)

Show the instructor: 1. `docker images` listing your `loan-default` image. 2. A `curl` to the containerized `/predict` returning JSON. 3. The app reachable on a **remapped** host port (e.g., 9000). 4. `docker logs` showing a request you just made.

"Break it" challenge — three ways to break a container

Break each on purpose, observe the failure, then fix it (work in your pair, one at a time):

1. **Missing `EXPOSE` / wrong port** — change the `CMD` port to `--port 8000` but keep running `-p 8080:8080`. Hit `:8080` → connection fails/hangs. **Why?** The app listens on 8000 inside, but you mapped 8080. Fix by matching the ports.
2. **Wrong `CMD`** — change `app.main:app` to `main:app`. Rebuild and run → container **crashes on start**. Read `docker logs` (`ModuleNotFoundError`). Fix the module path.
3. **Unpinned dependency** — in `requirements.txt` temporarily change `scikit-learn==1.5.2` to just `scikit-learn`, rebuild. It *might* pull a newer version. Discuss: why can an unpinned dep make a build that worked last week fail today — and how pinning gives you reproducible images? Restore the pin.

Interactive element — "It works on my machine!" swap

Pairs **export** their built image and hand it to another pair:

```
docker save loan-default -o loan-default.tar     # pair A
```

Pair B loads and runs it **without any of pair A's source code**:

```
docker load -i loan-default.tar                  # pair B
docker run -p 8080:8080 loan-default
curl http://127.0.0.1:8080/health
```

Debrief: the whole point of containers is that "it works on my machine" becomes "it works on *every* machine." The image carried Python, deps, code, and the model together.

Common pitfalls

- **`Cannot connect to the Docker daemon`** — Docker Desktop isn't running. Start it and wait.
- **`COPY failed: model.joblib not found`** — you didn't run `train.py`, or `.dockerignore` excludes it. The image needs the model file present at build time.
- **`Bind for 0.0.0.0:8080 failed: port is already allocated`** — another container/app uses 8080. Use a different host port (`-p 8081:8080`) or stop the other container.
- **Editing code doesn't change the container** — images are snapshots. You must `docker build` again after code changes.
- **Build is painfully slow every time** — you changed `requirements.txt` (busts the cache) or you edited code *above* the deps layer. Keep `COPY requirements.txt` before code.

- **App unreachable despite mapping** — you forgot `--host 0.0.0.0` (localhost inside the container is invisible from outside). Our starter already sets this; don't remove it.

LAB 4 · DAY 3

Lab 4 — CI/CD with GitHub Actions

Maps to: Session 7 (Day 3 — Automation / Deploy / Monitoring) **Estimated time:** 2 hours **Format:** live-coding + pair programming + "turn CI red" challenge

Objective

Automate quality checks so nothing broken reaches `main`. By the end you will:

- Push the project to a **public GitHub repo**.
- Have a **GitHub Actions** workflow that **lints**, **runs pytest**, adds a **model accuracy threshold test**, and **builds the Docker image** — automatically, on every push.
- Read a CI run, understand red vs green, and fix a failing pipeline.

What you'll build

A `.github/workflows/ci.yml` pipeline (a starter already exists) that gates every push, plus a new `tests/test_model.py` that fails the build if the model is too inaccurate.

Setup

- A free **GitHub account**, and **git** installed (`git --version`).
- The starter workflow is already at `.github/workflows/ci.yml`.

```
cd sample-project
source .venv/bin/activate
git --version
```

Step by step

Step 1 — Read the pipeline (live-code together)

Open `.github/workflows/ci.yml`. Identify the pieces with your pair:

```
on:               # WHEN it runs (push + PR to main)
jobs:
  test:           # job 1: lint + prepare data/model + pytest
  docker-build:   # job 2 (needs: test): build the image
```

> **CI = Continuous Integration:** every push runs the same checks in a clean cloud machine, so "works on my machine" can't hide a broken build. **CD** extends this to automatic deployment (Lab 5). GitHub Actions is **free for public repositories**.

Step 2 — Create the GitHub repo and push

Create a **public** repo on github.com named `loan-default-mlops` (no README, no .gitignore — we have our own). Then:

```
cd sample-project
git init
git add .
git commit -m "Initial MLOps starter project"
git branch -M main
git remote add origin https://github.com/<YOUR-USERNAME>/loan-default-mlops.git
git push -u origin main
```

On GitHub, click the **Actions** tab. You should see the **CI** workflow running. Watch the logs stream. Wait for the ✔ green check (or ✖ red).

Step 3 — Add a model quality test (your turn)

Unit tests check code; **model tests** check the *model's behaviour*. Create `tests/test_model.py`:

```
"""Model quality gates: the model must clear a minimum accuracy bar."""
import joblib
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

from make_data import make_dataset
from src.features import FEATURE_COLUMNS, TARGET_COLUMN

MIN_ACCURACY = 0.75   # CI fails if the model is worse than this


def test_model_meets_accuracy_threshold():
    df = make_dataset()
    X, y = df[FEATURE_COLUMNS], df[TARGET_COLUMN]
    _, X_test, _, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )
    model = joblib.load("model.joblib")
    acc = accuracy_score(y_test, model.predict(X_test))
    assert acc >= MIN_ACCURACY, f"Accuracy {acc:.3f} below threshold {MIN_ACCURACY}"


def test_high_risk_scores_higher_than_low_risk():
    """A behavioural test: an obviously risky applicant should score higher."""
    model = joblib.load("model.joblib")
    from src.features import row_to_frame
    low_risk = row_to_frame({"age": 45, "income": 120000, "loan_amount": 5000,
                             "credit_score": 800, "employment_years": 20,
                             "num_prior_defaults": 0})
    high_risk = row_to_frame({"age": 25, "income": 20000, "loan_amount": 40000,
                              "credit_score": 480, "employment_years": 0,
                              "num_prior_defaults": 4})
    p_low = model.predict_proba(low_risk)[0, 1]
    p_high = model.predict_proba(high_risk)[0, 1]
    assert p_high > p_low
```

Run locally first:

```
pytest -q
```

Then commit and push — CI runs it automatically:

```
git add tests/test_model.py
git commit -m "Add model quality gates to CI"
git push
```

Step 4 — Make lint actually enforce (tighten the pipeline)

The starter runs `ruff check . || true` — the `|| true` means lint failures are *ignored*. Find the `# TODO (LAB 4)` in `ci.yml` and remove `|| true`:

```
      - name: Lint (ruff)
        run: ruff check .
```

Push. If ruff finds issues, CI goes **red** — fix them (or run `ruff check --fix .` locally) until it passes. Now lint is a real gate.

Step 5 — Add a status badge (visible proof)

Copy the badge markdown from **Actions** → **CI** → ⋯ → **Create status badge** and paste it at the top of `README.md`:

```
![CI](https://github.com/<YOUR-USERNAME>/loan-default-mlops/actions/workflows/ci.yml/
badge.svg)
```

Push. Your README now shows a live green/red CI badge.

Checkpoint (raise your hand)

Show the instructor: 1. Your **public** GitHub repo with the code pushed. 2. A **green** CI run in the Actions tab that includes lint + pytest + your model tests + docker build. 3. The **CI badge** rendering on your README.

"Break it" challenge — turn CI red on purpose

1. **Break a test:** raise `MIN_ACCURACY` to `0.999` in `test_model.py`, commit, push. Watch CI go **red**. Open the failed run → find the exact failing assertion in the logs. Then lower it back to `0.75` and watch it go green again.
2. **Break the build:** introduce a syntax error in `app/main.py` (e.g., delete a closing paren), push. See which job fails and how the logs point to the line.
3. **Break lint:** add an unused import like `import os` at the top of a file with no use, push. If you removed `|| true`, ruff fails the job. Fix it.

Debrief: **a red build is a feature, not a failure** — CI caught the problem before your users did.

Interactive element — "Read the red log" race

The instructor pushes a commit with a *hidden* bug to a shared demo repo (or each pair breaks their own). First pair to correctly (a) name which **job** failed, (b) paste the **exact error line** from the logs, and (c) propose the **one-line fix** wins. Emphasis: reading CI logs is a core on-the-job skill.

Common pitfalls

- **`Permission denied (publickey)` / auth on push** — use an HTTPS remote and a **Personal Access Token** as the password, or set up the GitHub CLI (`gh auth login`).

- **Actions tab is empty** — the workflow file must live at exactly `.github/workflows/ci.yml` (check spelling/indent — YAML is whitespace-sensitive).
- **CI fails on `model.joblib` not found** — the pipeline runs `make_data.py` + `train.py` before tests; keep those steps. If you `.gitignore` the model, CI regenerates it (good).
- **Private repo eats free minutes** — keep the repo **public** for unlimited free Actions minutes.
- **Local passes, CI fails** — CI uses a clean machine and the *pinned* `requirements.txt`. If it differs from what you installed locally, align them. This is CI doing its job.
- **YAML indentation errors** — spaces only, never tabs; keep two-space indents consistent.

LAB 5 · DAY 3

Lab 5 — Cloud Deployment to a Free Host

Maps to: Session 8 (Day 3 — Automation / Deploy / Monitoring) **Estimated time:** 1.5 hours **Format:** live-deploy + pair programming + cold-start challenge

Objective

Take the **exact same Docker image** from Lab 3 and put it on the public internet — **for free**. By the end you will:

- Deploy the container to **Hugging Face Spaces (Docker Space)** — the primary path.
- Get a **public URL** and call live `/health` and `/predict` from anywhere.
- (Alternative) Deploy the same repo to a **Render free web service**.
- Understand `$PORT` injection and **cold starts** on free tiers.

What you'll build

A live, public API — e.g. `https://<your-username>-loan-default.hf.space` — that classmates (and your phone) can call.

Setup

- A free **Hugging Face account** (https://huggingface.co/join).
- Your working project from Labs 2–4, including a committed `model.joblib`.
- Confirm the model file exists and is committed:

```
cd sample-project
ls -lh model.joblib          # a few MB is fine for the free tier
git status                   # model.joblib should be tracked
```

> Free tiers are small. Keep the image lean (our `.dockerignore` already excludes `mlruns/`, tests, and docs).

Primary path — Hugging Face Spaces (Docker Space)

Step 1 — Create the Space

1. Go to **https://huggingface.co/new-space**.
2. **Owner:** you. **Space name:** `loan-default`.
3. **License:** any (e.g. `mit`). **SDK:** choose **Docker** → **Blank**.
4. **Visibility: Public** (free). Click **Create Space**.

HF creates a git repo for the Space with a `README.md` (containing YAML metadata) and a placeholder `Dockerfile`.

Step 2 — Tell HF which port to route to

HF routes traffic to the port declared in the Space `README.md` metadata (default `7860`). Our container listens on **8080**, so set `app_port: 8080`. Your Space `README.md` top should look like:

```
---
title: Loan Default
emoji: 
colorFrom: blue
colorTo: green
sdk: docker
app_port: 8080
pinned: false
---

# Loan Default Predictor
Live MLOps course demo. Try the API at `/docs`.
```

Step 3 — Push your project into the Space repo

Clone the Space repo and copy your project files into it (or add the Space as a second remote). The clean way:

```
# from a folder OUTSIDE sample-project
git clone https://huggingface.co/spaces/<your-username>/loan-default
cd loan-default

# copy the app, deps, model, and Dockerfile from your project
cp -r "../sample-project/app" .
cp -r "../sample-project/src" .
cp "../sample-project/requirements.txt" .
cp "../sample-project/Dockerfile" .
cp "../sample-project/make_data.py" "../sample-project/train.py" .
cp "../sample-project/model.joblib" .

# keep the HF README.md metadata from Step 2 (edit it if needed)
git add .
git commit -m "Deploy loan-default API to HF Spaces"
git push
```

> HF may ask you to authenticate. Use your HF **username** and an **access token** (Settings → Access Tokens → New token, role `write`) as the password. Large files: if git complains, run `git lfs install` — our model is small, so you likely won't need it.

Step 4 — Watch it build and get your URL

On the Space page, open the **Logs/Building** view. It runs your Dockerfile (installs deps, copies code). When it says **Running**, your public URL is:

```
https://<your-username>-loan-default.hf.space
```

Test it (from any terminal, or your phone browser):

```
curl https://<your-username>-loan-default.hf.space/health

curl -X POST https://<your-username>-loan-default.hf.space/predict \
  -H "Content-Type: application/json" \
  -d '{"age":35,"income":60000,"loan_amount":15000,"credit_score":700,"employment_years":
8,"num_prior_defaults":0}'
```

Open `https://<your-username>-loan-default.hf.space/docs` — your live Swagger UI. **Share the URL with your pair and have them call it.**

Alternative path — Render free web service

If HF is congested, use Render:

1. Sign up at **https://render.com** (free, GitHub login).
2. **New** → **Web Service** → connect your **public GitHub repo** from Lab 4.
3. **Runtime:** Docker (Render auto-detects your `Dockerfile`). **Instance type: Free**.
4. Render **injects a `$PORT`** env var automatically. Our `CMD` already reads `${PORT}` — so it just works, no change needed.
5. Click **Create Web Service**, watch the build, then hit the public `https://<service>.onrender.com/health`.

> Render's free service **spins down after ~15 min idle** → the first request afterwards is slow (**cold start**). That's the tradeoff for free.

Checkpoint (raise your hand)

Show the instructor: 1. Your **public URL** returning `{"status":"ok"}` on `/health`. 2. A **live** `/predict` call returning a probability (done from a terminal *or your phone*). 3. The live `/docs` page.

"Break it" challenge — port & cold start

1. **Wrong port:** on HF, change `app_port` in the Space `README.md` to `9999` (a port nothing listens on), push, and reload. The Space shows an error / never becomes healthy. Read the logs — HF is routing to a port with no server. Fix it back to `8080`. **Lesson:** the platform's port config MUST match the port your app binds.
2. **Missing `$PORT` handling (Render):** temporarily hardcode `--port 8080` in the `CMD` and redeploy on Render. Render's health check (which expects its injected `$PORT`) fails. Restore `${PORT}`. **Lesson:** cloud hosts choose the port; your app must obey `$PORT`.
3. **Cold start:** leave a Render deploy idle ~15 min, then time the first request (`time curl .../health`) vs. the second. Discuss: why free tiers sleep, and how a paid tier or a keep-alive ping would change this.

Interactive element — Class API Gallery + peer testing □

Every pair drops their **public URL** into the shared class board. Then each pair must: - Call **two other pairs'** live `/predict` endpoints, and - Try to make each one return a **422** (reusing your Lab 2 skills) against a *live* service.

Fastest pair to get all their peers' URLs responding wins bragging rights. Great "wow" moment: 30 students each have a real model live on the internet, for free.

Common pitfalls

- **Build succeeds but URL shows an error** — port mismatch. HF `app_port` must equal the port your `CMD` binds (8080 here).
- **`model.joblib` not found in the Space** — you didn't copy/commit it into the Space repo. It must be present at build time.

- **Push rejected / auth failed on HF** — use an **access token** (write role) as your password, not your account password.
- **Image too big / build times out** — you accidentally included `mlruns/`, `.venv/`, or datasets. Check `.dockerignore`.
- **First Render request hangs** — cold start; wait and retry. Not a bug.
- **CORS errors from a browser fetch** — for curl/Swagger you're fine; browser cross-origin calls need CORS middleware (out of scope, mention only).

LAB 6 · DAY 3

Lab 6 — Monitoring, Drift & Governance

Maps to: Session 9 (Day 3 — Monitoring, Drift & Governance; lecture + short hands-on) **Estimated time:** ~1 hour hands-on **Format:** live-coding + pair programming + "shift the distribution" challenge

Objective

A deployed model is not "done" — the world changes and the model silently rots. By the end you will:

- Understand **data drift** (the input distribution moves away from training).
- Compute two drift signals with **scipy**: the **KS test** and **PSI (Population Stability Index)**.
- **Simulate drift**, detect it, and **log a drift report**.
- Define a **retraining trigger** rule and note basic **governance**.

What you'll build

A `monitor.py` that compares "reference" (training) data against "current" (live) data, prints a per-feature drift table, and decides whether to fire a **retraining trigger**.

Setup

```
cd sample-project
source .venv/bin/activate
python make_data.py            # reference data = data/loan.csv
```

Concept primer (read with your pair): - **Reference distribution** = what the model was trained on. - **Current distribution** = what production is seeing now. - **KS test** (`scipy.stats.ks_2samp`): are two samples from the same distribution? Small **p-value** → they differ → drift. - **PSI**: a single number summarising how much a feature's distribution shifted. Rule of thumb: **< 0.1** no real shift · **0.1–0.25** moderate · **> 0.25** major drift.

Step by step

Step 1 — Create the drift monitor (live-code together)

Create `monitor.py`:

```
"""Data-drift monitor: compare reference vs current data per feature."""
from __future__ import annotations

import numpy as np
import pandas as pd
from scipy.stats import ks_2samp

from src.features import FEATURE_COLUMNS

PSI_WARN = 0.1
PSI_ALERT = 0.25
KS_P_ALERT = 0.05


def psi(reference: np.ndarray, current: np.ndarray, bins: int = 10) -> float:
    """Population Stability Index for one numeric feature."""
    # Fixed bin edges from the reference distribution (quantile bins).
    quantiles = np.linspace(0, 1, bins + 1)
    edges = np.unique(np.quantile(reference, quantiles))
    edges[0], edges[-1] = -np.inf, np.inf

    ref_counts, _ = np.histogram(reference, bins=edges)
    cur_counts, _ = np.histogram(current, bins=edges)

    ref_pct = np.clip(ref_counts / ref_counts.sum(), 1e-6, None)
    cur_pct = np.clip(cur_counts / cur_counts.sum(), 1e-6, None)

    return float(np.sum((cur_pct - ref_pct) * np.log(cur_pct / ref_pct)))


def drift_report(reference: pd.DataFrame, current: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for col in FEATURE_COLUMNS:
        ks_stat, ks_p = ks_2samp(reference[col], current[col])
        col_psi = psi(reference[col].to_numpy(), current[col].to_numpy())
        drifted = (col_psi > PSI_ALERT) or (ks_p < KS_P_ALERT)
        rows.append(
            {
                "feature": col,
                "psi": round(col_psi, 4),
                "ks_pvalue": round(ks_p, 4),
                "drifted": drifted,
            }
        )
    return pd.DataFrame(rows)


def retraining_trigger(report: pd.DataFrame, max_drifted: int = 1) -> bool:
    """Fire retraining if more than `max_drifted` features have drifted."""
    return int(report["drifted"].sum()) > max_drifted


if __name__ == "__main__":
    import sys

    reference = pd.read_csv("data/loan.csv")
    current = pd.read_csv(sys.argv[1] if len(sys.argv) > 1 else "data/loan.csv")
```

```
    report = drift_report(reference, current)
    print(report.to_string(index=False))
    n_drift = int(report["drifted"].sum())
    print(f"\nDrifted features: {n_drift}")
    if retraining_trigger(report):
        print("RETRAINING TRIGGER FIRED -> schedule a retrain.")
    else:
        print("No retraining needed.")
```

Step 2 — Baseline: current == reference (no drift)

```
python monitor.py data/loan.csv
```

Because current equals reference, PSI ≈ 0, KS p-values are high, `drifted` is `False` everywhere, and **no trigger** fires. This is your healthy baseline.

Step 3 — Simulate drift (create shifted "production" data)

Create `make_drift.py`:

```
"""Create a drifted copy of the data to simulate the world changing."""
import pandas as pd

df = pd.read_csv("data/loan.csv")

# Simulate an economic downturn: incomes fall, loan sizes rise, scores dip.
df["income"] = df["income"] * 0.75
df["loan_amount"] = df["loan_amount"] * 1.30
df["credit_score"] = (df["credit_score"] - 60).clip(300, 850)

df.to_csv("data/loan_drifted.csv", index=False)
print("Wrote data/loan_drifted.csv")
```

```
python make_drift.py
python monitor.py data/loan_drifted.csv
```

Now `income`, `loan_amount`, and `credit_score` show **high PSI** and **tiny KS p-values** → `drifted = True`, and the **RETRAINING TRIGGER FIRES**. You just detected drift.

Step 4 — Log the drift signal (tie back to MLflow)

Monitoring results should be **tracked**, just like training. Append to `monitor.py`'s `__main__`:

```
    import mlflow
    mlflow.set_tracking_uri("sqlite:///mlflow.db")
    mlflow.set_experiment("loan-default-monitoring")
    with mlflow.start_run(run_name="drift-check"):
        for _, r in report.iterrows():
            mlflow.log_metric(f"psi_{r['feature']}", r["psi"])
            mlflow.log_metric(f"ksp_{r['feature']}", r["ks_pvalue"])
        mlflow.log_metric("n_drifted", n_drift)
```

Re-run against the drifted data, then open the MLflow UI (`mlflow ui --backend-store-uri sqlite:///mlflow.db`) and view the `loan-default-monitoring` experiment. You now have a **time series of drift** you could chart.

Step 5 — Governance note (discuss, 5 min)

With your pair, jot one sentence each for: - **Model card:** what does this model do, on what data, with what known limits? - **Audit trail:** MLflow gives you *which* data/params/metrics produced *which* registered version — that's your paper trail for a regulator or an incident review. - **Human in the loop:** a fired trigger shouldn't auto-ship a new model to prod — it should open a ticket for review, then promote via the Registry stages (Lab 1).

Checkpoint (raise your hand)

Show the instructor: 1. `monitor.py` on the **baseline** → no trigger. 2. `monitor.py` on the **drifted** data → drift detected + **trigger fired**. 3. A `loan-default-monitoring` run in the MLflow UI with logged PSI/KS metrics.

"Break it" challenge — shift one feature until it trips

1. Edit `make_drift.py` to shift **only one** feature by a **small** amount (e.g. `credit_score - 10`). Does it trip the KS test but not PSI, or neither? Find the *smallest* shift that flips `drifted` to `True` for that feature.
2. Now shift a feature the model barely uses vs. one it relies on heavily. Discuss: **drift in an important feature matters more** than drift in a weak one — statistical drift ≠ performance drop.
3. Tune the trigger: set `max_drifted=0` (any drift retrains) vs `max_drifted=3` (very tolerant). Which is right for a bank? Argue both sides.

Interactive element — "Drift or Noise?" prediction game □

Before running the monitor, the instructor projects a `make_drift.py` with a *secret* transformation. Each pair **predicts** which features will be flagged `drifted` and writes it down. Reveal the report. Score 1 point per correct feature. Then discuss surprises — often a small mean shift on a high-variance feature *doesn't* trip, while a tight feature does. Builds intuition for real monitoring.

Common pitfalls

- **`FileNotFoundError: data/loan.csv`** — run `python make_data.py` first.
- **PSI is `inf` or `nan`** — a bin had zero counts; the `np.clip(..., 1e-6, None)` in `psi()` guards against this. Don't remove it.
- **KS says "drift" on identical data** — you compared different-sized samples or reloaded the wrong file; verify you passed the right CSV path.
- **Confusing data drift with concept drift** — data drift = inputs change; concept drift = the input→label relationship changes. Both need monitoring; this lab covers data drift.
- **Treating any statistical drift as an emergency** — set thresholds and a trigger rule; not every wiggle needs a retrain.

CAPSTONE

Capstone & Mock Technical Interview

Maps to: Session 10 (Day 3 — final) **Estimated time:** ~1.5 hours **Format:** live demo + peer review + mock technical interview

Objective

Prove you can take a model **end to end** and talk about it like an engineer. You'll demo your **live, deployed** loan-default service and defend your design choices in a short mock interview — the same questions a real MLOps/ML-engineer screen would ask.

What you'll present

The full pipeline you built across three days:

```
train → track (MLflow) → serialize (joblib) → serve (FastAPI)
      → containerize (Docker) → CI/CD (GitHub Actions) → deploy (free host) → monitor
(drift)
```

Part A — Deliverables checklist (have these ready)

Tick every box before you present:

- [ ] **Public GitHub repo** (from Lab 4) with all code and a **green CI badge**.
- [ ] **MLflow evidence** (Lab 1): screenshot of the Compare view (3+ runs) and the Model Registry showing a **versioned** model.
- [ ] **FastAPI service** (Lab 2): `/health`, `/predict`, `/predict_batch` working; validation returns 422 on bad input.
- [ ] **Dockerfile** (Lab 3): image builds and runs locally.
- [ ] **CI pipeline** (Lab 4): lint + pytest + **model accuracy threshold** + docker build, all green.
- [ ] **Live public URL** (Lab 5): `/health` and `/predict` respond from the internet.
- [ ] **Drift monitor** (Lab 6): `monitor.py` detects your simulated drift and fires the retraining trigger; metrics logged to MLflow.
- [ ] **README** explaining setup and each component.

Part B — Demo (5 minutes per pair, strictly timed)

Follow this script so every demo covers the whole pipeline:

1. **(30s) Pitch:** "This predicts loan default from 6 features. Here's the architecture." (Show a one-line diagram.)
2. **(60s) Train + track:** show the MLflow Compare view; point to your **registered** best model and its version/stage.
3. **(90s) Live API:** open your **public URL** `/docs`. Run one **valid** prediction and one that returns **422**. Bonus: call it from your phone.
4. **(60s) CI/CD:** show the green Actions run and the badge; explain what would happen on a failing test.
5. **(60s) Monitoring:** run `monitor.py` on drifted data live; show the trigger firing.
6. **(30s) One thing you'd do next** (e.g., auth, autoscaling, a real dataset, model card).

Peer scoring (each pair scores the presenting pair, 1–5 each): *works end-to-end*, *clarity*, *handles the 422 gracefully*, *drift detection*, *stage presence*.

Part C — Mock technical interview (10–12 min per student)

The instructor (and a peer "shadow interviewer") ask **4–5** of these. Answer as if in a real screen — be specific, reference *your* project.

Fundamentals 1. What problem does MLOps solve that plain ML training doesn't? 2. Walk me through your pipeline from raw data to a live prediction. 3. What is *train/serve skew* and what in your code prevents it? *(Hint:* `src/features.py` *.)*

Experiment tracking 4. Why track experiments? What exactly did you log, and how would you reproduce a past result? 5. Difference between the MLflow tracking store and the Model Registry? What are model *stages* for?

Serving & packaging 6. Why FastAPI + Pydantic over a bare Flask endpoint? Show me a 422 and explain why it's good. 7. Why containerize? What does your Docker image contain that a `pip install` on my laptop wouldn't guarantee? 8. Why does your `CMD` read `$PORT` instead of hardcoding a port?

CI/CD 9. What runs in your CI, and why gate merges on it? What's a *model test* vs a *unit test*? 10. Your CI is red. Walk me through how you'd diagnose it.

Deploy & monitor 11. You deployed free — what are the tradeoffs (cold starts, resources) vs. a paid host? 12. What is data drift? How did you detect it, and what should fire a retrain? 13. A regulator asks "why did the model reject this applicant six months ago?" How does your setup help you answer?

Curveballs (pick one) 14. Your live model's accuracy dropped 10% this week but no code changed. First three things you check? 15. How would you roll back to a previous model version in production?

Sample strong answer (for #3, so students calibrate)

> "Train/serve skew is when the features at serving time don't match training — wrong columns, wrong order, or different preprocessing. In my project both `train.py` and the API import `FEATURE_COLUMNS` and `row_to_frame()` from `src/features.py`, so there's one source of truth. The scaler is inside the sklearn `Pipeline`, so preprocessing is serialized *with* the model — the API can't accidentally skip it."

Interactive element — rotating interview panels □

Students rotate in triads: one **interviewee**, one **interviewer** (asks from the bank above), one **scribe** (notes one strong answer + one to improve). Rotate every ~4 minutes so everyone plays all three roles. Debrief as a class: the 3 most common weak spots, and the best answer heard.

Grading rubric (suggested, 100 pts)

AREA	PTS
End-to-end works (train → live URL)	30
MLflow tracking + registered/versioned model	15
API quality (validation, batch, docs)	10
Docker image builds & runs	10
Green CI with model test	10

AREA	PTS
Live public deployment	10
Drift detection + trigger	10
Interview: clarity & correctness	15
(cap at 100)	

Common pitfalls (day-of)

- **Live URL asleep** (Render cold start) — hit `/health` a minute before you present to warm it.
- **Demoing localhost** instead of the public URL — the capstone requires the *deployed* endpoint.
- **No 422 to show** — prepare a bad payload in advance.
- **Overrunning 5 minutes** — rehearse once; the script above fits the time.
- **Can't explain your own choices** — the interview weighs *understanding* over polish. Know why each piece exists.

LECTURE ACTIVITIES

Lecture Activities — Interactive Exercises for Sessions 1, 2, 4 & 9

These keep the **lecture** sessions active instead of slide-only. Each is 10–20 minutes, needs no setup beyond the shared class board, and ties directly into the labs. Run them *inside* the lecture, not as add-ons.

Materials: whiteboard or shared doc, sticky notes (or a chat channel), timer. Groups of 2–4.

Session 1 — Intro to MLOps

Activity 1.1 — "Why did this model fail in prod?" case cards (15 min)

Hand each group **one** case card (read aloud, then discuss). They must name (a) the **root cause**, (b) which **MLOps practice** would have caught it, and (c) which **lab** in this course addresses it.

Case cards: 1. *The vanishing accuracy.* A fraud model was 95% accurate in the notebook, 61% in production. Nobody can reproduce the notebook run — the data file was overwritten. → *tracking/reproducibility (Lab 1)*. 2. *Works on my machine.* The model runs on the data scientist's laptop but crashes on the server: `sklearn` version mismatch. → *pinned deps + containers (Lab 3)*. 3. *The silent rot.* A demand model was great at launch, terrible six months later. No one was watching. → *drift monitoring (Lab 6)*. 4. *The 3 a.m. deploy.* An engineer pushed a "tiny fix" straight to prod on Friday; it broke predictions all weekend. No tests ran. → *CI/CD gates (Lab 4)*. 5. *The mystery columns.* Serving code sent features in a different order than training. Predictions were garbage but the API returned 200 OK. → *train/serve skew (Labs 2 & 6)*.

Debrief: each group posts their MLOps practice on the board. Build the class's shared definition: *MLOps = making ML reproducible, testable, deployable, and observable.* Preview how the course's one project hits every point.

Activity 1.2 — Maturity self-rating poll (5 min)

Quick hands/poll: "For your last ML mini-project, did you (a) version the data, (b) track experiments, (c) test the model, (d) deploy it, (e) monitor it?" Tally on the board. Almost all hands drop by (d). That gap **is** this course.

Session 2 — Data & Feature Management

Activity 2.1 — Train/serve-skew spot-the-bug (15 min)

Project this snippet. Groups race to find **all** the ways training and serving can disagree:

```
# TRAINING
feature_cols = ["age", "income", "loan_amount", "credit_score",
                "employment_years", "num_prior_defaults"]
X = df[feature_cols]
scaler = StandardScaler().fit(X)          # scaler fit here...
model.fit(scaler.transform(X), y)

# SERVING (a different file, written weeks later)
features = [req.income, req.age, req.loan_amount,   # <-- order swapped!
            req.credit_score, req.employment_years, req.num_prior_defaults]
pred = model.predict([features])          # <-- scaler NOT applied!
```

Bugs to find: (1) `age` / `income` order swapped; (2) the scaler is never applied at serving; (3) the feature list is duplicated by hand, so they can drift apart. **Fix:** one shared `FEATURE_COLUMNS` + a shared transform, and put the scaler *inside* a `Pipeline` so it's serialized with the model. Point them to `src/features.py` — that's exactly why it exists.

Activity 2.2 — Feature-store card sort (10 min)

Give groups cards: `raw event log`, `age`, `avg_purchases_last_30d`, `one-hot country`, `user_id`, `credit_score`, `timestamp`. Sort into **raw data** vs **engineered feature** vs **identifier (don't train on it!)**. Discuss why leaking an ID or a post-outcome field causes *data leakage* — a model that looks brilliant offline and useless live.

Session 4 — Model Serialization & Packaging

Activity 4.1 — Serialization pitfalls sort (15 min)

Groups sort these statements into **TRUE / FALSE / "depends"** and justify:

1. "A pickled/joblib model is safe to load from any source." → **FALSE** (arbitrary code execution; only load models you trust).
2. "If I saved the model but not the sklearn version, loading may break later." → **TRUE** (version skew) — hence pinned `requirements.txt`.
3. "Saving just the model weights is enough to serve predictions." → **DEPENDS** (you also need the preprocessing — put it in the `Pipeline`).
4. "joblib is generally better than pickle for large NumPy arrays." → **TRUE**.
5. "The model file alone documents what inputs it expects." → **FALSE** (you need the feature schema — e.g., our Pydantic model + `FEATURE_COLUMNS`).

Debrief: what a *deployable artifact* really is = model **+** preprocessing **+** input schema **+** pinned environment. Tie to Lab 2/3.

Activity 4.2 — "Package the model" whiteboard relay (10 min)

Each group draws what must ship inside the Docker image to serve predictions. Reveal the answer by opening the real `Dockerfile`: base image, pinned deps, `src/` + `app/`, and `model.joblib`. Anything they forgot is a future prod incident.

Session 9 — Monitoring, Drift & Governance

Activity 9.1 — Drift scenario discussion (15 min)

Present scenarios; groups decide **drift? which kind? retrain or not?**

1. A new marketing campaign brings in much younger applicants than before. → *data drift on `age`; likely retrain.*
2. A law changes what counts as a "default." → *concept drift (label meaning changed); definitely retrain + relabel.*
3. Applications drop 5% one week, same mix of people. → *volume change, not distribution drift; don't retrain.*
4. A logging bug sends `credit_score` in a 0–1 scale instead of 300–850. → *not real drift — a data pipeline bug; fix the pipeline, don't retrain!*

Key takeaway: a drift alarm is a **question, not an order** — investigate before retraining. Ties directly to Lab 6's trigger rule.

Activity 9.2 — Governance role-play (10 min)

Assign roles: **data scientist**, **ML engineer**, **compliance officer**, **product manager**. A model just rejected a loan and the applicant complains. Each role states what they need: the DS wants the run that produced the model; the engineer wants the deployed version + inputs; compliance wants an audit trail and a model card; the PM wants a customer-facing explanation. Debrief: the MLflow Registry + tracking store is what lets you answer all four — governance isn't paperwork, it's traceability.

Facilitation tips

- **Timebox hard** — use a visible timer; cut discussion at the buzzer and debrief.
- **Make them commit** — write an answer on the board *before* you reveal, so they can't hedge.
- **Always bridge to the lab** — end each activity with "you'll do this for real in Lab N."
- **Reward participation** — the class leaderboard from Lab 1 can carry points across these too.